hefty sum to join a course in a private institute to master that. Instead, focus on the fundamentals first. For example, a good grasp of data structures and algorithms is important to get any entry-level job. Similarly, focus on honing your C skills if you're not too good at it.

Once you have a reasonably good grounding in the fundamentals, find out what area in computing motivates you and what you would like to do in the future. In other words, you should choose an area that you're passionate about. It may be network programming, device driver development, Web programming, distributed computing, or database administration. Don't limit yourself, and it often takes time to figure out what kind of IT job you would like, particularly if you're not from an IT background. For example, if you're from the electronics field and love playing with resistors and capacitors (i.e., you have fun with circuit design), you might like low-level device driver programming. If you love reading books and have good writing skills, you might find technical writing to be interesting. Most students have a misconception that getting a job in the IT industry means a programming job that involves writing code. It's not at all true. There are a wide range of jobs that you can consider—as a tester, a configuration engineer, a Web designer, a technical writer, or a database administrator. If you have specialised or niche skills that are relevant to the industry, it's easy to find a job. If you don't like any of the IT jobs, and hate computers, perhaps you should look out for a job in an area that you like. Think about it: if you get an IT job that you hate, what's good in that job for you in the long run? Perhaps, you